

Lab 02: Socket Programming

Duration: 2 hours | In Lab Evaluation & Assignment

Tools: Python 3 (`socket`), `netstat` / `ss`

Objectives

By the end of this lab, students will be able to:

- Describe UDP's connectionless, unreliable datagram service.
- Implement a UDP echo server and client in Python, with error handling, timeouts, retries, and basic concurrency.
- Use Python's `logging` module to capture runtime events.
- Explain the TCP three-way handshake and connection teardown.
- Write a TCP echo server and client in Python using `SOCK_STREAM` sockets.
- Demonstrate persistence of a TCP connection across multiple send/receive calls.
- Observe TCP connection states using `netstat` or `ss`.
- Configure inter-PC communication within the lab network and parameterize client/server addresses.

Lab Description

Part A: UDP Echo Server

1. In `udp_server.py`, import `socket` and configure the server:

```
1 # Import the socket module to create UDP sockets
2 import socket
3
4 # Create a UDP socket (AF_INET = IPv4, SOCK_DGRAM = UDP)
5 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
6
7
8 # Bind the socket to all interfaces on port 5005
9 # '0.0.0.0' makes the server reachable from any network it's
   attached to
10
11 sock.bind(('0.0.0.0', 5005))
12 print("UDP server listening on port 5005...")
```

2. Implement the receive-and-echo loop with error handling:

```
1 try:
2     # Enter an infinite loop to receive and echo packets
3     while True:
4         # recvfrom(1024) waits for up to 1024 bytes from any
           client
5         data, addr = sock.recvfrom(1024)
6         # Print the raw data and the client address (IP, port)
```

```

7         print(f"Received {data!r} from {addr}")
8         # Send the exact same data back to the sender
9         sock.sendto(data, addr)
10    except KeyboardInterrupt:
11        print("\nShutting down server.")
12        sock.close()

```

Part B: UDP Client

1. In `udp_client.py`, import modules and set up the socket:

```

1  # Import socket for networking and time for RTT measurements
2  import socket, time
3
4  # Create a UDP socket
5  sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
6
7  # Set a 1 second timeout for blocking socket operations
8  # This is the time which we will wait for the echo
9  sock.settimeout(1.0)
10
11 # Define server address as localhost on port 5005
12 # localhost -> own machine -> 127.0.0.1
13 server_addr = ('localhost', 5005)

```

2. Send a single message and receive echo:

```

1  #---- Single message exchange ----
2  message = 'Hello, UDP Server'
3  # Encode the string to bytes and send to the server
4  sock.sendto(message.encode('utf-8'), server_addr)
5  try:
6      # Wait for up to 1024 bytes response
7      data, _ = sock.recvfrom(1024)
8      print("Echo:", data)
9  except socket.timeout:
10     print("No response, server may be down.")

```

3. Extend to multiple messages with retry:

```

1  #---- Multiple messages with retry and RTT ----
2  rtts = [] # to store round-trip times
3
4  # sending 5 messages with max 3 retries
5  for i in range(5):
6      text = f"Msg {i}".encode('utf-8')
7      attempts = 0
8      while attempts < 3:
9          start = time.time() # start RTT timer
10         sock.sendto(text, server_addr) # send datagram
11         try:
12             data, _ = sock.recvfrom(1024) # wait for echo
13             elapsed = time.time() - start

```

```

14         rtts.append(elapsed)
15         print(f"Response {i}:", data, f"(RTT={elapsed:.3f}s)")
16         break
17     except socket.timeout:
18         attempts += 1
19         print(f"Retry {attempts} for message {i}")
20     time.sleep(0.5) # brief pause between messages

```

Part C: Testing and Analysis

1. Run Server:

- Terminal 1: `python3 udp_server.py`

2. Run Client:

- Terminal 2: `python3 udp_client.py`

3. Observe:

- Server console logs each datagram and client address.
- Client console shows echoes or timeout/retry messages.

4. Discussion:

- Characteristics of UDP (no handshake, no ordering/delivery guarantee).
- Behavior when server is offline (no ACKs, silent drops).
- How timeouts and retries improve reliability in UDP applications.

Part D: Concurrency

Till now, the server would have processed the messages from more than one clients (if there were multiple clients) in a sequential manner. So, if multiple clients send messages to the server, it can't process them at the same time. To make it happen, we have to use threading. In simple terms, threading gives us the ability to divide the process (in our case the server.py) into multiple threads (different methods can be different threads) which can run concurrently. You can find details in this link - [Multi-threading](#). In our case, we can use handle requests as a thread so that if there are multiple clients, there can be multiple threads handling those clients at the same time. Below is the modified server code to handle multiple clients at the same time -

```

1 import socket
2 import threading
3
4 # Create UDP server socket
5 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
6 # Allow address reuse
7 sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
8
9 sock.bind(('0.0.0.0', 5005))
10 print("UDP server listening on port 5005...")
11
12 # Handler for each incoming message

```

```

13 def handle_request(data, addr):
14     print(f"Received {data!r} from {addr}")
15     sock.sendto(data, addr)
16
17 try:
18     while True:
19         data, addr = sock.recvfrom(1024)
20         # Spawn a new thread per request for concurrent handling
21         # target specifies the callable (function or method) that the
22         # new thread will execute
23         # A tuple of positional arguments to pass into target. In this
24         # case, once the thread begins, it effectively does:
25         # handle_request (data, addr)
26         # daemon ensures threads won't prevent program exit
27
28         threading.Thread(target=handle_request, args=(data, addr),
29                           daemon=True).start()
30 except KeyboardInterrupt:
31     print("\nShutting down server.")
32     sock.close()

```

You can achieve the same outcome in java using the following code

```

1  import java.net.*;
2  import java.io.*;
3
4  public class UdpEchoServer {
5      public static void main(String[] args) {
6          final int PORT = 5005;
7          DatagramSocket socket = null;
8          try {
9              socket = new DatagramSocket(null);
10             socket.setReuseAddress(true);
11             socket.bind(new InetSocketAddress("0.0.0.0", PORT));
12             System.out.println("UDP server listening on port " + PORT
13                               + "...");
14
15             byte[] buffer = new byte[1024];
16             while (true) {
17
18                 DatagramPacket packet = new DatagramPacket(buffer,
19                                                             buffer.length);
20                 socket.receive(packet);
21
22                 new Thread(() -> {
23                     try {
24                         String received = new String(packet.getData(),
25                                                         0, packet.getLength());
26                         InetAddress address = packet.getAddress();
27                         int port = packet.getPort();
28                         System.out.printf("Received '%s' from %s:%d\n",
29                                         received, address, port);

```

```

27         DatagramPacket response = new DatagramPacket(
28             packet.getData(), packet.getLength(),
29             address, port);
30         socket.send(response);
31     } catch (IOException e) {
32         System.err.println("Error handling request: "
33             + e.getMessage());
34         e.printStackTrace();
35     }
36     }, "Handler-Thread").start();
37 }
38 } catch (SocketException se) {
39     System.err.println("Socket error: " + se.getMessage());
40     se.printStackTrace();
41 } catch (IOException ioe) {
42     System.err.println("I/O error: " + ioe.getMessage());
43     ioe.printStackTrace();
44 } finally {
45     if (socket != null && !socket.isClosed()) {
46         socket.close();
47         System.out.println("Server socket closed.");
48     }
49 }
50 }

```

Part E: Logging in python

The logging module allows you to capture runtime events with timestamps, severity levels, and flexible output destinations.

- Basic configuration

```

1 import logging
2 logging.basicConfig(
3     filename='server.log',
4     level=logging.INFO,
5     format='%(asctime)s - %(message)s',
6     datefmt='%Y-%m-%d %H:%M:%S'
7 )

```

- Writing log entries

```

1 logging.info("Alice registered from ('127.0.0.1', 54321)")
2 logging.error("Failed to forward message")

```

- Use debug(), info(), warning(), error(), critical() for different severities.

Part F: TCP Echo Server

1. Create a new Python file `tcp_echo_server.py`.
2. In the file, import the socket module:

```
1 import socket
```

3. Build a TCP server socket that:

- Binds to all interfaces on port 6006.
- Listens for a single incoming connection.
- Accepts the connection and prints the client's address.
- In a loop, receives up to 1024 bytes, echoes it back, and prints each message.
- Closes the connection cleanly when the client disconnects.

4. Example skeleton:

```
1 # --- SERVER SIDE ---
2
3 # 1. Create a new socket object using IPv4 and TCP
4 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5
6 # 2. Bind the socket to all network interfaces ('0.0.0.0') on port
   6006
7 server.bind(('0.0.0.0', 6006))
8
9 # 3. Put the socket into listening mode, with a backlog of 5
   queued connection
10 server.listen(5)
11
12 # 4. Block and wait for an incoming connection; when one arrives,
13 #     accept() returns a new socket object (conn) and the client
   address (addr)
14 conn, addr = server.accept()
15
16 # 5. Print the address of the newly connected client
17 print("Connected by", addr)
18
19 # 6. Enter an infinite loop to continually receive and echo data
20 while True:
21     # 7. Read up to 1024 bytes from the client; blocks until data
       arrives
22     data = conn.recv(1024)
23
24     # 8. If no data was received, it means the client closed the
       connection - break out
25     if not data:
26         break
27
28     # 9. Print what was received for debugging/logging
29     print("Received:", data)
30
31     # 10. Send the exact same bytes back to the client
32     conn.sendall(data)
33
34 # 11. Close the client socket once done
```

```
35 conn.close()
```

5. Save and run the server:

```
$ python3 tcp_echo_server.py
```

Part G: TCP Client

1. Create `tcp_echo_client.py`.
2. In the file, import `socket` and:
 - Create a TCP socket.
 - Connect to `localhost` on port 6006.
 - Send a greeting (e.g. `b"Hello, TCP"`).
 - Receive and print the echoed data.
3. Example snippet:

```
1 # --- CLIENT SIDE ---
2
3 # 1. Create a new socket object using IPv4 and TCP
4 client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5
6 # 2. Connect this socket to the server running on localhost port
7     6006
8 client.connect(('127.0.0.1', 6006))
9
10 # 3. Loop over a list of byte strings to send
11 for msg in [b"Hello, TCP!", b"Message 2", b"Final msg"]:
12     # a) Send the entire message to the server (blocks until
13         sent)
14     client.sendall(msg)
15
16     # b) Wait for up to 1024 bytes of response from the server (
17         the echo)
18     data = client.recv(1024)
19
20     # c) Print out what the server echoed back
21     print("Echo:", data)
22
23 # 4. Close the client socket when all messages have been sent and
24     echoed
25 input("Press ENTER here to close the connection and exit...")
26 client.close()
```

4. Run the client while the server is listening.

Part H: Observing TCP States

1. While both server and client are running, open a separate terminal.

2. Run:
on windows:

```
C:\> netstat -an | findstr :6006
```


or on Linux:

```
$ ss -tn sport = :6006
```
3. Identify the line showing the connection in the ESTABLISHED state.
4. After the client closes, rerun the command to observe the socket transition (e.g. TIME_WAIT or CLOSE_WAIT).

Part I: Communicating within the lab

As you already know, each of our lab PCs are connected with one another through a central switch. To make one PC act as the TCP echo server and all others as clients, follow these detailed steps:

1. **Choose the Server Machine and Assign IP Address**
 - (a) On the designated server PC, open a terminal (Linux/macOS) or Command Prompt (Windows).
 - (b) Check its current IP address:
 - **Windows:** `ipconfig`
 - **Linux/macOS:** `ip addr show` or `ifconfig`
 - (c) If the network uses DHCP, note the assigned IP (e.g. 192.168.1.10). Otherwise, set a static IP in the same subnet as the clients (e.g. 192.168.1.10/24).
2. **Configure Firewall on server machine to Allow TCP Port 6006**
 - (a) **Windows:**
 - Open *Windows Defender Firewall* then go to *Advanced Settings*.
 - Create a new *Inbound Rule* allowing TCP port 6006.
 - (b) **Linux (using ufw):**

```
1 sudo ufw allow 6006/tcp
2 sudo ufw reload
```
3. **Run the TCP Echo Server on the Server PC**
 - (a) Ensure `tcp_echo_server.py` is present.
 - (b) Launch the server:

```
$ python3 tcp_echo_server.py
```
 - (c) The server will bind to `0.0.0.0:6006` and wait for incoming connections.
4. **Update and Run Clients on Each Student PC**

- (a) On every client PC, open `tcp_echo_client.py` in a text editor.
- (b) Modify the `connect()` line to point at the server's IP:

```
1 client.connect(('192.168.1.10', 6006))
```

(Replace 192.168.1.10 with your actual server IP.)

- (c) Save and run the client:

```
$ python3 tcp_echo_client.py
```

- (d) Observe each client printing three Echo: ... lines.

5. Verify TCP Connection States

- (a) While clients are connected and exchanging messages, open a new terminal.

- (b) On Linux:

```
$ ss -tn sport = :6006
```

- (c) On Windows:

```
C:\> netstat -an | findstr :6006
```

- (d) Confirm you see lines with `ESTABLISHED` between 192.168.1.10:6006 and each client's IP.
- (e) After clients exit, rerun the command to observe `TIME_WAIT` or `CLOSE_WAIT` states.

6. Optional: Parameterize Server IP for Flexibility

- (a) Edit `tcp_echo_client.py` to accept the server IP as a command-line argument:

```
1 import sys, socket
2
3 server_ip = sys.argv[1]          # Usage: python3 tcp_echo_client.
   py 192.168.1.10
4 client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5 client.connect((server_ip, 6006))
```

- (b) Run with:

```
$ python3 tcp_echo_client.py 192.168.1.10
```